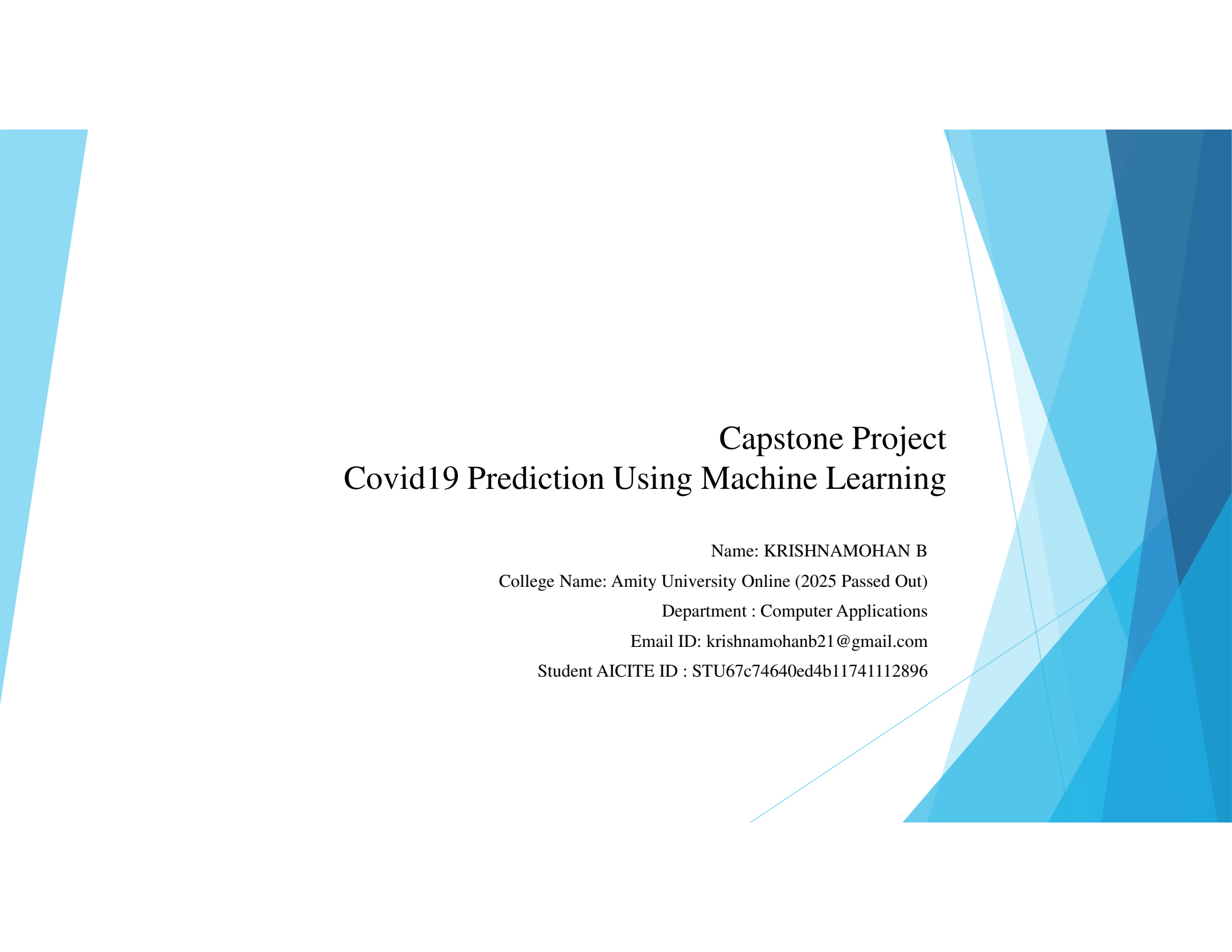




Capstone Project Covid19 Prediction Using Machine Learning

Name: KRISHNAMOHAN B

College Name: Amity University Online (2025 Passed Out)

Department : Computer Applications

Email ID: krishnamohanb21@gmail.com

Student AICITE ID : STU67c74640ed4b11741112896

Contents

- ▶ Problem Statement of Covid 19 Prediction Using Machine Learning
- ▶ Proposed System of Covid19 Prediction Using Machine Learning
- ▶ System Approach
- ▶ Algorithm and Deployment
- ▶ Results
- ▶ Conclusion
- ▶ Future Scope
- ▶ References

Problem Statement of Covid 19 Prediction Using Machine Learning

The outbreak of COVID-19 has created a global health crisis, affecting millions of people worldwide. Early detection of infected individuals plays a crucial role in preventing the spread of the virus and ensuring timely medical intervention.

Traditional diagnostic methods such as RT-PCR tests require laboratory facilities, time, and medical resources. In situations where testing resources are limited, there is a need for an intelligent system that can assist in predicting the likelihood of COVID-19 infection based on symptoms, medical history, and exposure factors.

This project aims to develop a Machine Learning model that predicts whether a person is COVID-19 positive or negative using the following input features:

Symptoms:

Breathing Problem, Fever, Dry Cough, Sore Throat, Running Nose, Headache, Fatigue, Gastrointestinal Issues

Pre-existing Medical Conditions:

Asthma, Chronic Lung Disease, Heart Disease, Diabetes, Hypertension

Exposure & Travel History:

Abroad Travel, Contact with COVID Patient, Attended Large Gathering, Visited Public Exposed Places, Family Working in Public Exposed Places

Preventive Measures:

Wearing Masks, Sanitization from Market

The target variable is:

COVID-19 (Positive / Negative)

Proposed System of Covid19 Prediction Using Machine Learning

The proposed system aims to develop a Machine Learning-based predictive model to determine whether a person is COVID-19 positive or negative based on symptoms, medical conditions, exposure history, and preventive measures.

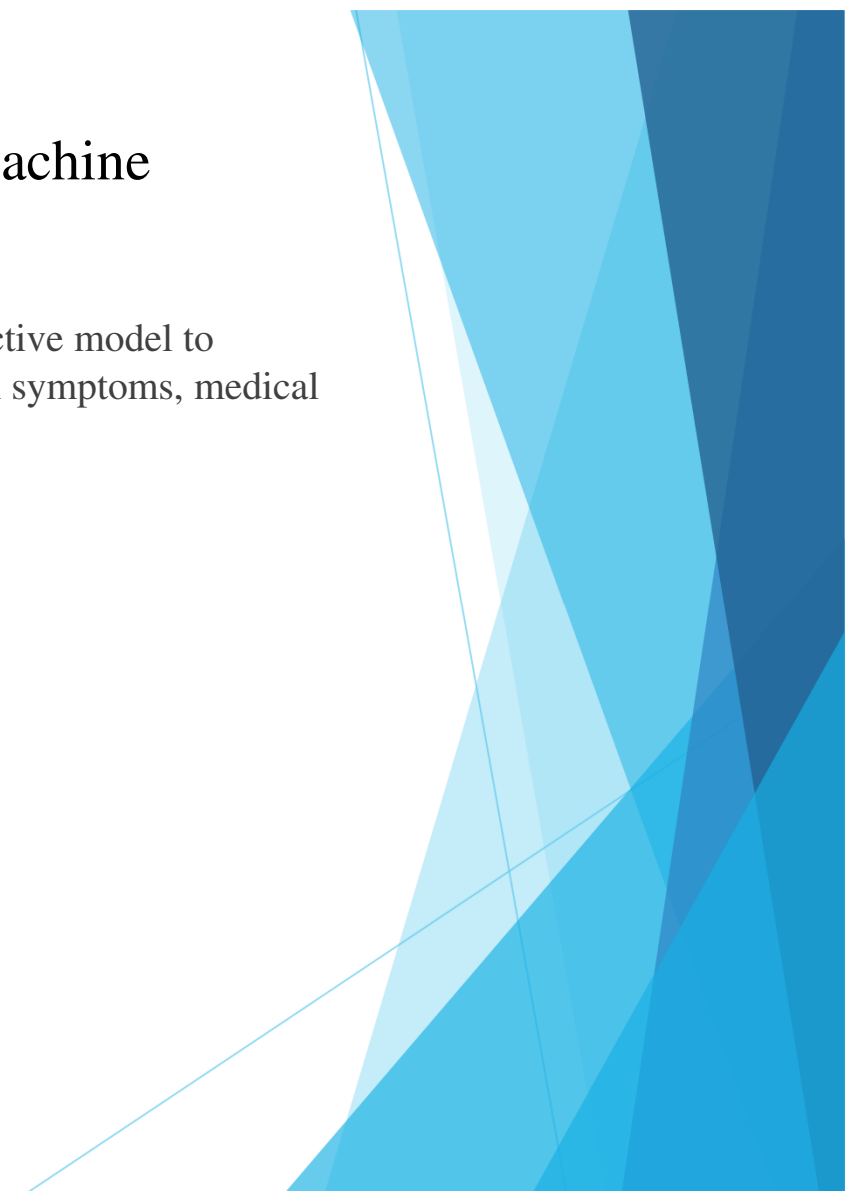
The dataset contains binary values:

0 = No

1 = Yes

The target variable:

COVID-19 (0 = Negative, 1 = Positive)



1) Data Collection

The dataset is collected in CSV format.

Each record represents an individual with the following features:

Symptoms

Breathing Problem

Fever

Dry Cough

Sore Throat

Running Nose

Headache

Fatigue

Gastrointestinal



Pre-existing Medical Conditions

Asthma

Chronic Lung Disease

Heart Disease

Diabetes

Hypertension

Exposure & Travel History

Abroad Travel

Contact with COVID Patient

Attended Large Gathering

Visited Public Exposed Places

Family Working in Public Exposed Places

Preventive Measures

Wearing Masks

Sanitization from Market

Target

COVID-19 (Positive / Negative)



2) Data Preprocessing

Since the dataset uses binary values (0 and 1):

No categorical encoding is required.

Check for missing or null values.

Split dataset into:

80% Training Data

20% Testing Data

3) Machine Learning Algorithm

Classification algorithms are used to predict COVID-19 status:

1) Logistic Regression

2) Decision Tree

3) Random Forest

4) Support Vector Machine (SVM)

5) K-Nearest Neighbors (KNN)

The selected model learns patterns between symptoms, exposure factors, and infection status.

The best-performing algorithm is chosen based on evaluation metrics.



4) Deployment

(User Interface – Flask)

The trained model is deployed using Flask web framework.

Working:

User enters 0 (No) or 1 (Yes) for each feature.

Flask application receives input.

Input data is converted into numerical format.

Trained ML model predicts output.

Result is displayed as:

COVID-19 Positive

COVID-19 Negative

The system provides a simple and user-friendly interface for prediction.



5) Evaluation

The model is evaluated using:

- 1) Accuracy
- 2) Precision
- 3) Recall
- 4) F1-Score
- 5) Confusion Matrix

These metrics measure how effectively the model predicts COVID-19 cases.

6) Result

The final system successfully predicts whether a person is COVID-19 positive or negative based on input features.

The model provides:

- 1) Fast prediction
- 2) Preliminary screening support
- 3) Assistance in early risk identification

System Approach

HARDWARE REQUIREMENTS : The hardware requirements for running this website and model are:

RAM – 8.00 GB

Operating System – Windows 11

Processor – Intel(R) Core(TM) i3-1115g4

Processor speed – 3.00 GHz

SOFTWARE REQUIREMENTS: The programming language used to develop this application is Python and the IDE used is

Jupyter Notebook. Front end is made using HTML and is integrated with flask.

Programming Language – Python

Python IDE – Jupyter Notebook

Python Libraries: Flask

Libraries to build the model

- 1) Pandas
- 2) matplotlib
- 3) seaborn
- 4) sklearn
- 5) Flask

Algorithm and Deployment

Algorithm

- 1) Logistic Regression
- 2) Decision Tree Classifier
- 3) Support Vector Machine
- 4) Random Forest Classifier
- 5) K Neighbors Classifier

Logistic Regression

This is the one of the most common model used in ML ,Logistic Regression is often applied in the actual manufacturing context the fields such as data mining, automatic disease diagnosis and economic prediction. For our model, use Logistic regression to know the risk factors for heart disease and forecast the probability of disease occurrence based on risk factors. This model is most frequently applied for classification, primarily two-category issues (that is, there are only two types of output, each representing one category), and can indicate the probability of occurrence of each classification event. Logistic regression model is shown below: This technique used is also known as sigmoid function .Sigmoid function helps in the easy representation in graphs. Logistic regression also provides better accuracy. By using equation the logistic regression algorithm is represented in the graphs showing the difference between the attributes.



Where Y refers to binary dependent variable (Y is equal to 1 if event happens; Y=0 otherwise), e stands for the foundation of natural logarithms and Z means

$$prob(Y = 1) = \frac{e^z}{1 + e^z}$$

with constant β_0 , coefficients β_j and predictors X_j , for p predictors ($j=1,2,3,\dots,p$)

$$Z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$$

with constant β_0 , coefficients β_j and predictors X_j , for p predictors ($j=1,2,3,\dots,p$)

The process of modeling the probability of a discrete outcome given an input variable is known as the Logistic Regression. The most common logistic regression, as its name suggests is not regression rather it is a classification algorithm that classifies something that can take two values such as true/false, yes/no, and so on. Logistic regression identifies a hyperplane in a manner that when it passes through a function whose value ranges between 0 and 1 (typically we use sigmoidal), it optimizes cost function. Based on closeness to 0 or 1, it predicts a Boolean output. Here vector parameters is used for training. $\sigma(\cdot)$ is usually a sigmoid function, with output between 0 and 1.

Features of Logistic Regression

Multinomial logistic regression is the type of regression which uses the softmax function to compute probabilities.

- We use loss function to learn weights(vector w and bias b) from a labeled training. we perform such activity to minimize the cross-entropy loss.
- Iterative algos like gradient descent are used to get the weight(optimal).while minimizing the loss function the type of convex optimization problem.
- To avoid overfitting regularization is used.
- Logistic regression has the ability to transparently study the importance of individual features.

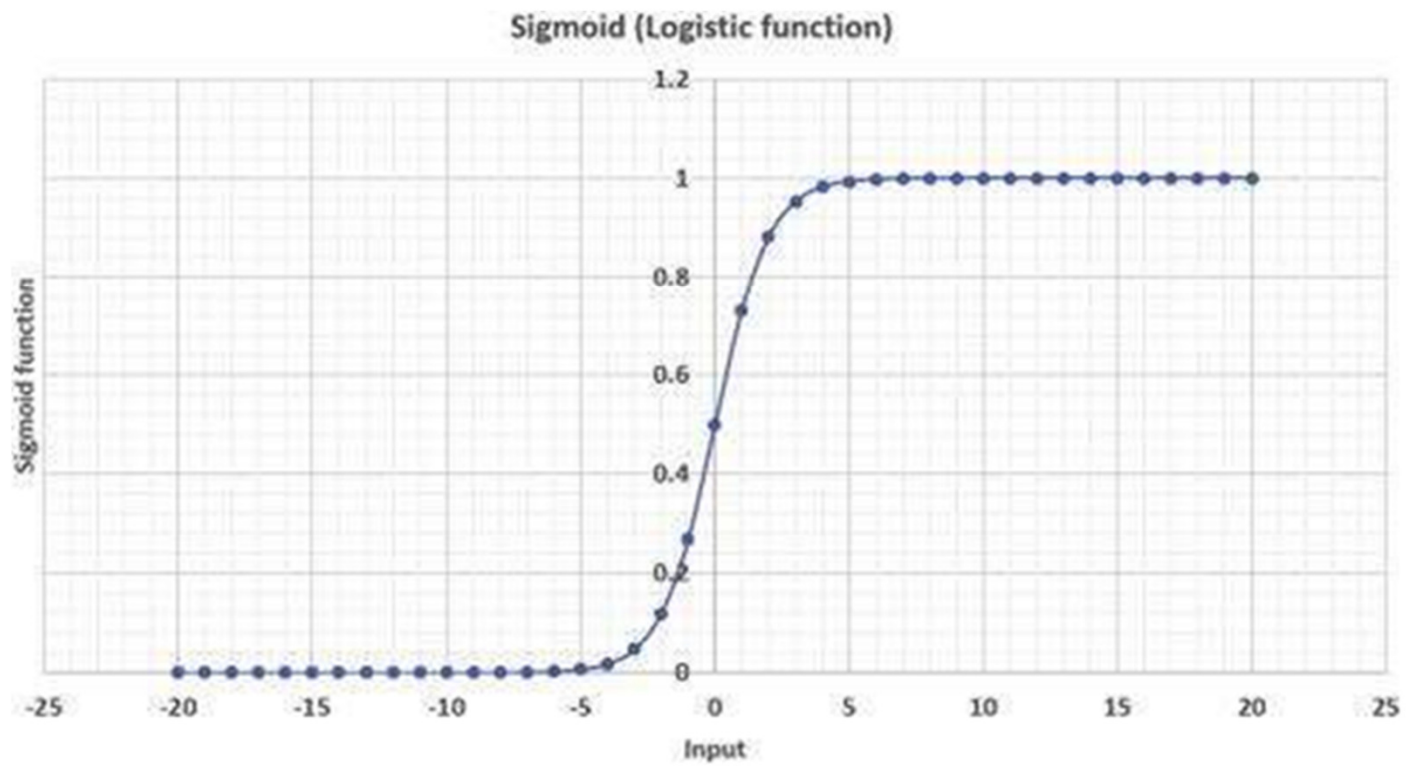
Advantages and Disadvantages of Logistic Regression

Advantages

- This technique performs well and fast where we have to classify unknown records.
- This is not limited to binary classification; we can easily extend it to multinomial regression.

Disadvantages

- Logistic regression will not perform well if the number of observations is lesser than the number of features; in such a condition, it may lead to overfitting.
- Logistic regression constructs linear boundaries. In the logistic function equation, x is the input variable. Let's feed in values -20 to 20 into the logistic function. As illustrated in Figure, the inputs have been transferred to between 0 and 1 .



▼ Logistic Regression

```
[72]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
```

```
[72]: ▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=1000)
```

```
[77]: y_pred = lr.predict(X_test)
```

```
[43]: from sklearn.metrics import confusion_matrix, classification_report
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

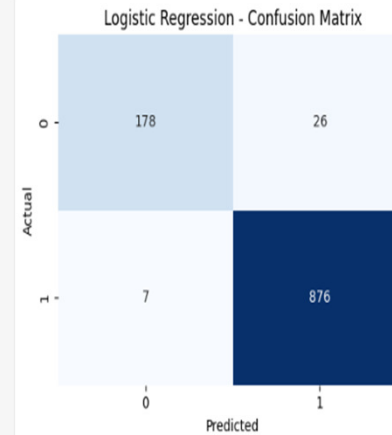
# Plot Confusion Matrix
plt.figure(figsize=(5,4))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            cbar=False)
plt.xlabel("Predicted")
plt.ylabel("Actual")

# Calculate overall metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

plt.title("Logistic Regression - Confusion Matrix")
plt.tight_layout()
plt.show()

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, zero_division=0))

# Overall metrics
print("Overall Model Performance:")
print("Accuracy :", round(accuracy * 100, 2), "%")
print("Precision :", round(precision * 100, 2), "%")
print("Recall :", round(recall * 100, 2), "%")
print("F1-Score :", round(f1 * 100, 2), "%")
```



Classification Report:

	precision	recall	f1-score	support
0	0.96	0.87	0.92	204
1	0.97	0.99	0.98	883
accuracy			0.97	1087
macro avg	0.97	0.93	0.95	1087
weighted avg	0.97	0.97	0.97	1087

Overall Model Performance:

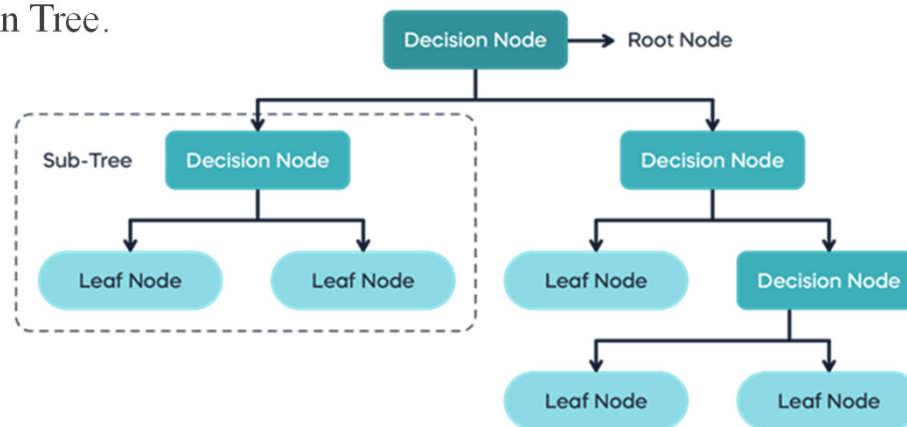
Accuracy : 96.96 %
Precision : 96.95 %
Recall : 96.96 %
F1-Score : 96.91 %

Accuracy 96.96 %

Decision Tree Classifier

A decision tree contains a flowchart-like structure. In the structure of DT(Decision Tree) in a test, an attribute is represented using an internal node. The result of the test is represented using the branch. To represent a class label, a leaf node is used. To represent classification rules paths from root to leaf are used. This is the type of analysis in which closely related influence diagrams are used for a visual and analytical root. Where the expected values of competing alternatives are calculated.

Architecture of Decision Tree.



Decision Tree

```
[44]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
```

```
[44]: ▼ DecisionTreeClassifier ⓘ ⓘ
DecisionTreeClassifier(random_state=42)
```

```
[45]: y_pred = dt.predict(X_test)
```

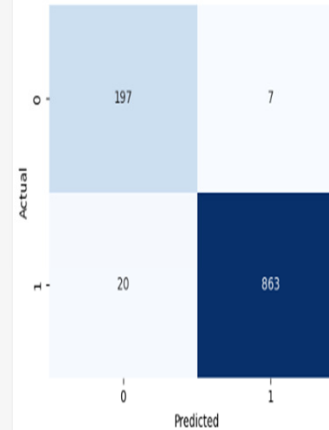
```
[46]: from sklearn.metrics import confusion_matrix, classification_report
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5,4))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            cbar=False)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Decision Tree - Confusion Matrix")
plt.tight_layout()
plt.show()

# 📄 Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, zero_division=0))

# 📄 Overall metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

print("Overall Model Performance:")
print("Accuracy :", round(accuracy * 100, 2), "%")
print("Precision :", round(precision * 100, 2), "%")
print("Recall :", round(recall * 100, 2), "%")
print("F1-Score :", round(f1 * 100, 2), "%")
```

Decision Tree - Confusion Matrix



Classification Report:

	precision	recall	f1-score	support
0	0.91	0.97	0.94	204
1	0.99	0.98	0.98	883
accuracy				1087
macro avg	0.95	0.97	0.96	1087
weighted avg	0.98	0.98	0.98	1087

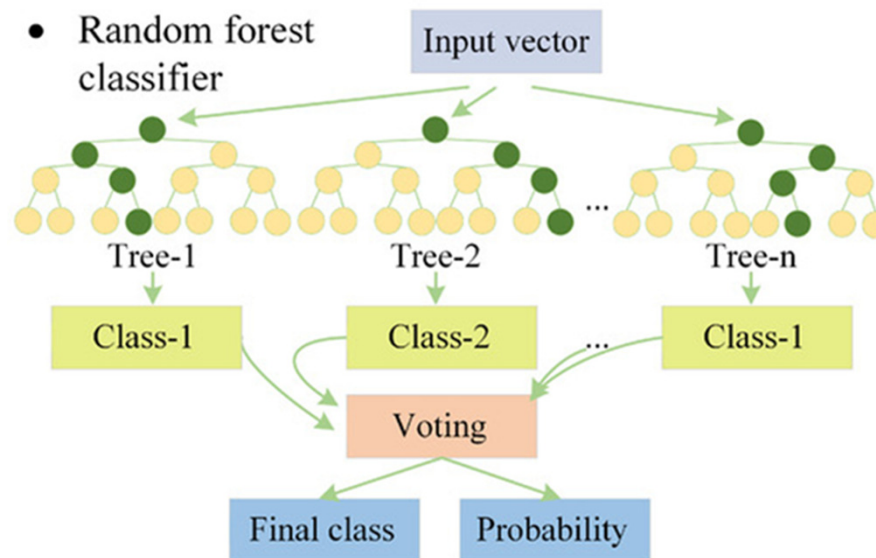
Overall Model Performance:

Accuracy : 97.52 %
Precision : 97.62 %
Recall : 97.52 %
F1-Score : 97.55 %

Accuracy 97.52 %

Random Forest Classifier

A Random Forest Classifier is a method of ensemble learning applied to classification problems. It constructs multiple decision trees and combines their predictions to enhance accuracy and avoid overfitting. Each tree is trained on a random subset of data and attributes, hence making the model stable, robust, and effective in handling complex datasets.



Features of Random Forest

- It runs Efficiently in scenarios where database is very huge.
- We can perform classification on Thousands of Input variables.
- Using this we can find which variable is useful for our classification.
- Using this we can easily calculate the missing data and also maintain the good accuracy.

Even though it has missing data.

Random Forest

```
[47]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

```
[47]: ▼ RandomForestClassifier ⓘ ⓘ
RandomForestClassifier(random_state=42)
```

```
[48]: y_pred = rf.predict(X_test)
```

```
[49]: from sklearn.metrics import confusion_matrix, classification_report
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns

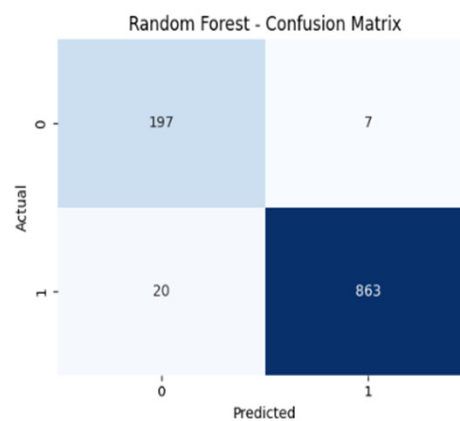
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Plot Confusion Matrix
plt.figure(figsize=(5,4))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            cbar=False)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Random Forest - Confusion Matrix")
plt.tight_layout()
plt.show()

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, zero_division=0))

# Calculate overall metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

# Print Overall Metrics
print("Overall Model Performance:")
print("Accuracy  :", round(accuracy * 100, 2), "%")
print("Precision :", round(precision * 100, 2), "%")
print("Recall    :", round(recall * 100, 2), "%")
print("F1-Score  :", round(f1 * 100, 2), "%")
```



Classification Report:

	precision	recall	f1-score	support
0	0.91	0.97	0.94	204
1	0.99	0.98	0.98	883
accuracy			0.98	1087
macro avg	0.95	0.97	0.96	1087
weighted avg	0.98	0.98	0.98	1087

Overall Model Performance:

Accuracy : 97.52 %
Precision : 97.62 %
Recall : 97.52 %
F1-Score : 97.55 %

Accuracy 97.52 %

Support Vector Machine

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It works by finding the optimal hyperplane that best separates data into different classes. The main goal of SVM is to maximize the margin between different classes while minimizing classification error.

Features of SVM

1. **Margin Maximization:** SVM finds the hyperplane that maximizes the margin between classes, which helps improve generalization and robustness.
2. **Support Vectors:** Only a few important data points (called support vectors) are used to define the hyperplane, making the model efficient.
3. **Effective in High Dimensions:** SVM performs well even when the number of features is very large (high-dimensional spaces).
4. **Kernel Trick:** Allows SVM to solve non-linear classification problems by transforming the input data into higher-dimensional space using kernel functions (e.g., RBF, polynomial).
5. **Versatile:** Can be used for binary as well as multi-class classification problems.
6. **Regularization:** SVM includes a regularization parameter (C) that balances the trade-off between maximizing the margin and minimizing the classification error.
7. **Robust to Overfitting (in high-dimensional space):** Due to its focus on margin and support vectors, SVM can generalize well, especially when data is not too noisy.

Support Vector Machine

```
[50]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix
svm = SVC(kernel='rbf', random_state=42)
svm.fit(X_train, y_train)
```

```
[50]: SVC
SVC(random_state=42)
```

```
[51]: y_pred = svm.predict(X_test)
```

```
[52]: from sklearn.metrics import confusion_matrix, classification_report
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns

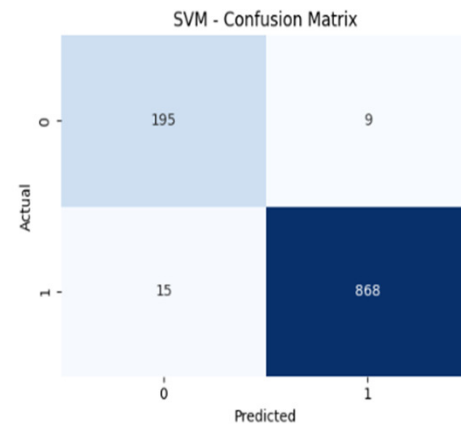
# 1 Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# 2 Plot Confusion Matrix
plt.figure(figsize=(5,4))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            cbar=False)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("SVM - Confusion Matrix")
plt.tight_layout()
plt.show()

# 3 Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, zero_division=0))

# 4 Calculate Overall Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

# 5 Print Overall Metrics
print("Overall Model Performance:")
print("Accuracy :", round(accuracy * 100, 2), "%")
print("Precision :", round(precision * 100, 2), "%")
print("Recall   :", round(recall * 100, 2), "%")
print("F1-Score :", round(f1 * 100, 2), "%")
```



Classification Report:

	precision	recall	f1-score	support
0	0.93	0.96	0.94	204
1	0.99	0.98	0.99	883
accuracy			0.98	1087
macro avg	0.96	0.97	0.96	1087
weighted avg	0.98	0.98	0.98	1087

Overall Model Performance:

Accuracy : 97.79 %
Precision : 97.83 %
Recall : 97.79 %
F1-Score : 97.8 %

Accuracy 97.79 %

K Neighbors Classifier

K Neighbors is a classification and regression online (lazy) learning algorithm. It's a non-parametric approach to the extent that it doesn't make any assumption regarding the distribution of the data.

How It Works:

Input Data: To predict a given new data point, KNN computes the similarity between this data point and the entire training data set.

Distance Metric: Any distance metric such as Euclidean, Manhattan, or others can be employed based on the problem.

Finding Neighbors: It finds the k training samples nearest to the input point.

Prediction: For classification, the most common label among the k neighbors is returned. For regression, the mean value of the k neighbors is taken as the prediction.

Model Training & Prediction (Using sklearn):

Training: Employ the `fit()` function of K Neighbors Classifier or KNeighbors Regressor from sklearn .neighbors.

Prediction: Employ the `predict()` function on the test set.

Selection of k value:

Selecting k (number of neighbors) is important:

A low k may produce overfitting (high variance).

A high k can smoothen the decision boundary but may lead to underfitting (high bias).

For selection of the best k , vary k and check the model performance using cross-validation.

The location at which adding k no longer further enhances accuracy appreciably is referred to as the knee point.

Kneighbour Classifier

```
[53]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
```

```
[53]: ▼ KNeighborsClassifier ⓘ ⓘ
KNeighborsClassifier()
```

```
[54]: y_pred = knn.predict(X_test)
```

```
[55]: from sklearn.metrics import confusion_matrix, classification_report
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns

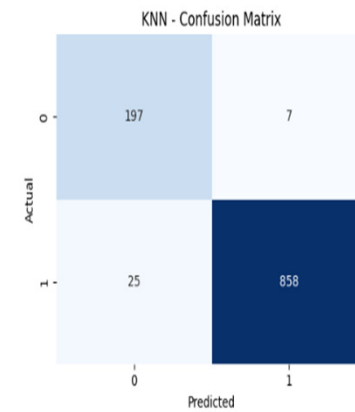
# 1 Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# 2 Plot Confusion Matrix
plt.figure(figsize=(5,4))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            cbar=False)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("KNN - Confusion Matrix")
plt.tight_layout()
plt.show()

# 3 Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, zero_division=0))

# 4 Calculate Overall Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

# 5 Print Overall Metrics
print("Overall Model Performance:")
print("Accuracy :", round(accuracy * 100, 2), "%")
print("Precision :", round(precision * 100, 2), "%")
print("Recall :", round(recall * 100, 2), "%")
print("F1-Score :", round(f1 * 100, 2), "%")
```



Classification Report:

	precision	recall	f1-score	support
0	0.89	0.97	0.92	204
1	0.99	0.97	0.98	883
accuracy			0.97	1087
macro avg	0.94	0.97	0.95	1087
weighted avg	0.97	0.97	0.97	1087

Overall Model Performance:

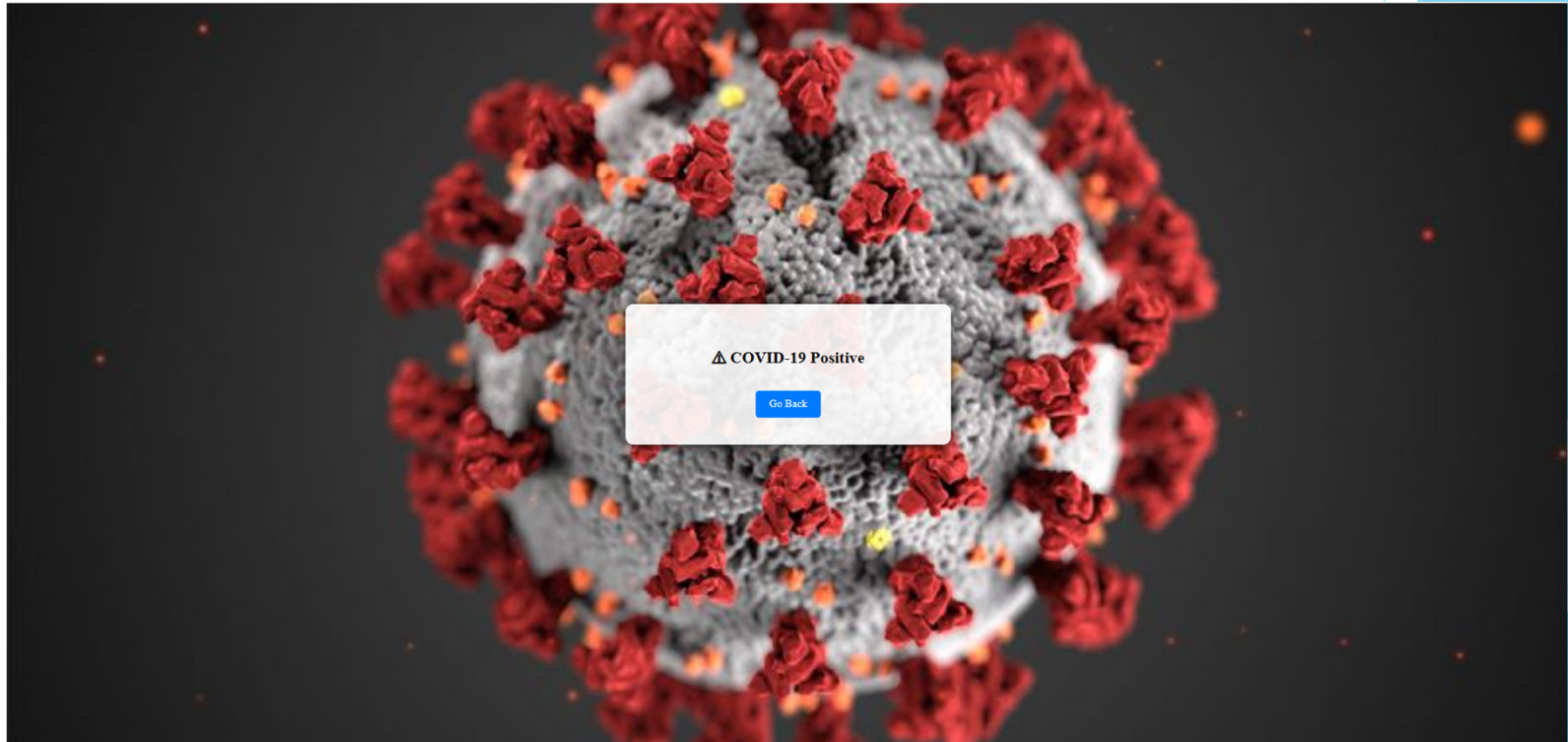
Accuracy : 97.06 %
Precision : 97.23 %
Recall : 97.06 %
F1-Score : 97.1 %

Accuracy : 97.06 %

Deployment (Working Model Diagram)

COVID-19 Prediction System
Enter 1 = Yes, 0 = No

Breathing Problem	1
Fever	1
Dry Cough	1
Sore Throat	1
Running Nose	1
Asthma	0
Chronic Lung Disease	0
Headache	0
Heart Disease	0
Diabetes	1
Hypertension	1
Fatigue	1
Gastrointestinal	1
Abroad Travel	0
Contact with COVID Patient	1
Attended Large Gathering	0
Public Places	1
Family Public Exposure	1
Wearing Masks	0
Sanitization from Market	0

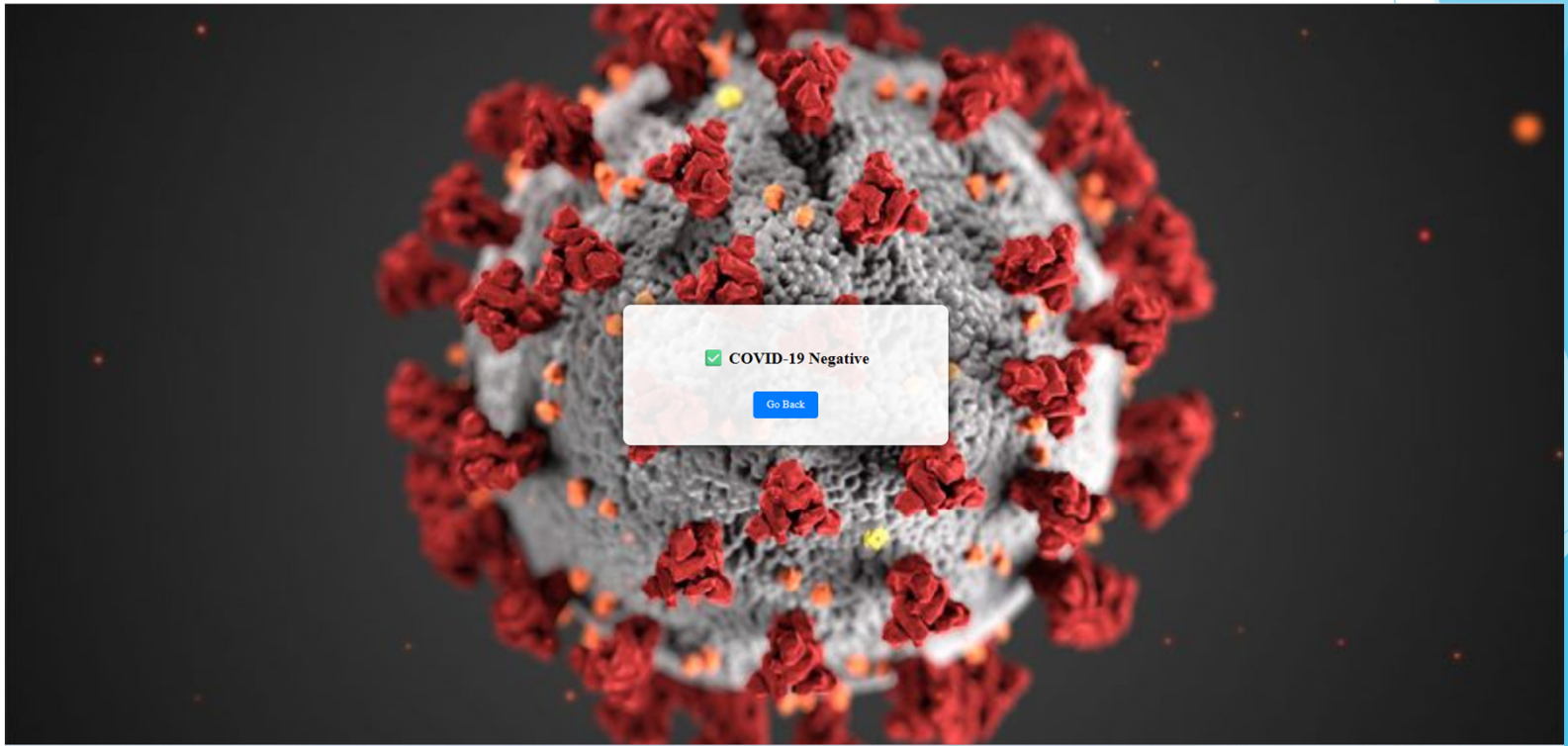


COVID-19 Prediction System

Enter 1 = Yes, 0 = No

Breathing Problem	1
Fever	1
Dry Cough	1
Sore Throat	0
Running Nose	1
Asthma	0
Chronic Lung Disease	0
Headache	1
Heart Disease	1
Diabetes	0
Hypertension	0
Fatigue	0
Gastrointestinal	0
Abroad Travel	0
Contact with COVID Patient	0
Attended Large Gathering	0
Public Places	0
Family Public Exposure	0
Wearing Masks	0
Sanitization from Market	0

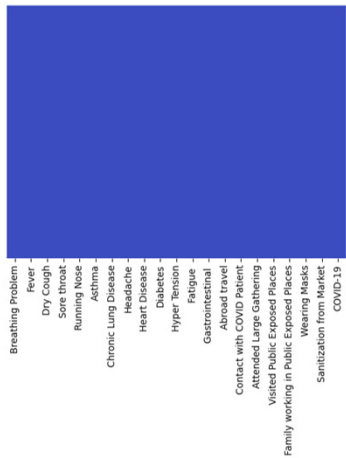
Predict



Results

```
[13]: import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(df.isnull(),yticklabels=False,cbar=False,cmap='coolwarm')
```

[13]: <Axes: >

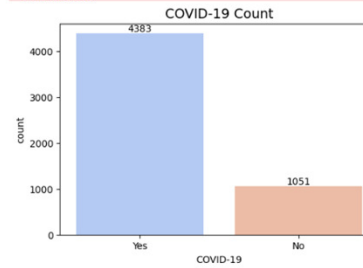


```
[19]: import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(6,4))
sns.countplot(
    x='COVID-19',
    data=df,
    palette='coolwarm'
)
for p in plt.gca().patches:
    height = p.get_height()
    plt.annotate(f'{int(height)}',
                (p.get_x() + p.get_width() / 2., height),
                ha='center',
                va='bottom',
                fontsize=10,
                color='black')
plt.title("COVID-19 Count", fontsize=14)
plt.show()
```

C:\Users\Krishnamohan B\AppData\Local\Temp\ipykernel_13264\1913693978.py:4: FutureWarning:

Passing "palette" without assigning "hue" is deprecated and will be removed in v0.14.0. Assign the 'x' variable to "hue" and set "legend=False" for the same effect.

```
sns.countplot(
```



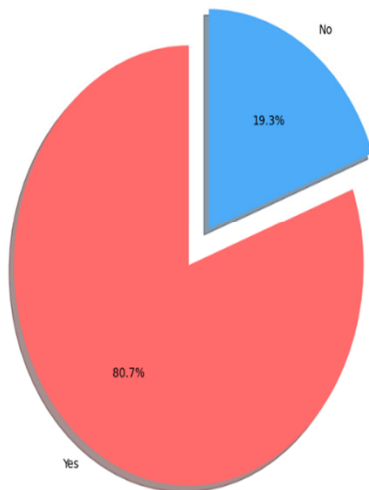
```
[23]: import matplotlib.pyplot as plt

plt.figure(figsize=(6,6))

df["COVID-19"].value_counts().plot.pie(
    explode=[0.1, 0.1],
    autopct="%1.1f%%",
    shadow=True,
    colors=['#ff6b6b', '#4dabf7'], # * Change colors here
    startangle=90
)

plt.title('COVID-19 Distribution', fontsize=14)
plt.ylabel("") # Removes default y-label
plt.axis('equal') # Makes circle perfect
plt.tight_layout()
plt.show()
```

COVID-19 Distribution



```
[25]: import matplotlib.pyplot as plt
import seaborn as sns

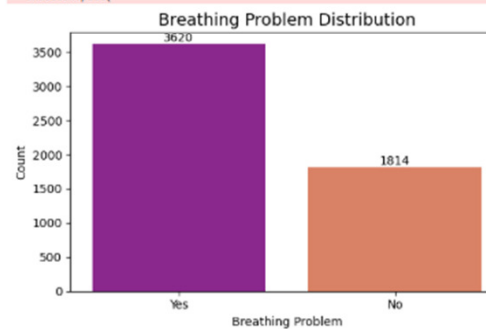
plt.figure(figsize=(6,4))

sns.countplot(
    x='Breathing Problem',
    data=df,
    palette="plasma"
)

# Add value labels on bars
for p in plt.gca().patches:
    height = p.get_height()
    plt.annotate(f'{int(height)}',
                (p.get_x() + p.get_width()/2., height),
                ha='center',
                va='bottom',
                fontsize=10,
                color='black')

plt.title("Breathing Problem Distribution", fontsize=14)
plt.xlabel("Breathing Problem")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

C:\Users\KrishnaMohan B\AppData\Local\Temp\ipykernel_13264\4158398715.py:6: FutureWarning:
 Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.



```
[26]: import matplotlib.pyplot as plt
import seaborn as sns

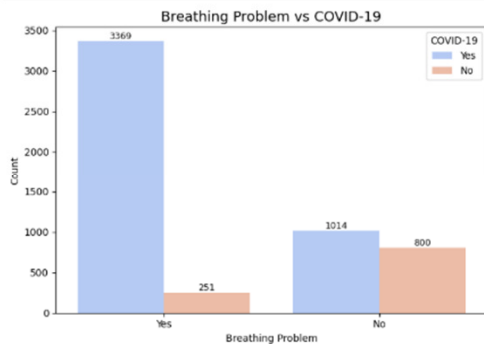
plt.figure(figsize=(7,5))

sns.countplot(
    x='Breathing Problem',
    hue='COVID-19',
    data=df,
    palette='coolwarm'
)

for p in plt.gca().patches:
    height = p.get_height()

    if height > 0:
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width() / 2., height),
            ha='center',
            va='bottom',
            fontsize=9,
            color='black')

plt.title("Breathing Problem vs COVID-19", fontsize=14)
plt.xlabel("Breathing Problem")
plt.ylabel("Count")
plt.legend(title="COVID-19")
plt.tight_layout()
plt.show()
```



```
[27]: import matplotlib.pyplot as plt
import seaborn as sns

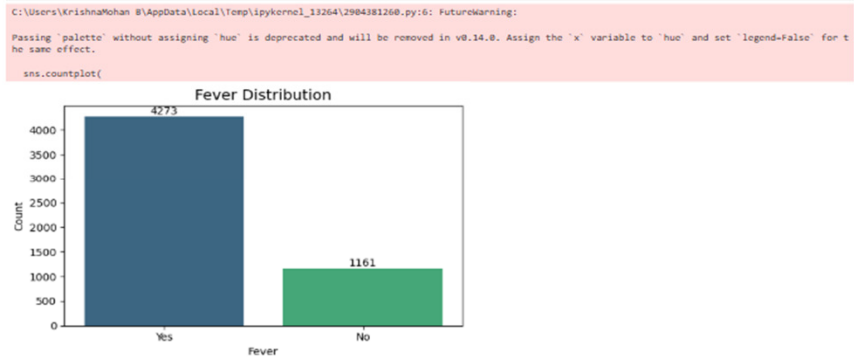
plt.figure(figsize=(6,4))

sns.countplot(
    x='Fever',
    data=df,
    palette='viridis'
)

# Add value labels on bars
for p in plt.gca().patches:
    height = p.get_height()

    if height > 0:
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width()/2., height),
            ha='center',
            va='bottom',
            fontsize=10,
            color='black')

plt.title("Fever Distribution", fontsize=14)
plt.xlabel("Fever")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```



```
[28]: import matplotlib.pyplot as plt
import seaborn as sns

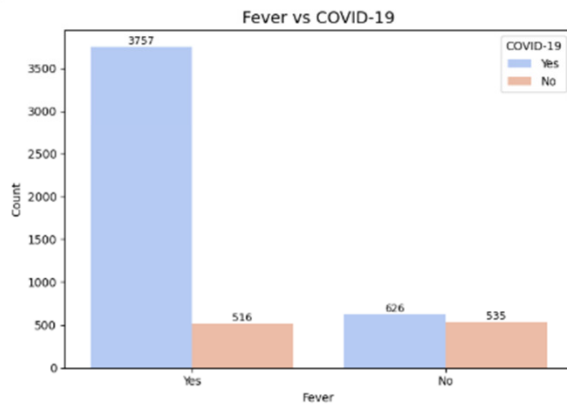
plt.figure(figsize=(7,5))

sns.countplot(
    x='Fever',
    hue='COVID-19',
    data=df,
    palette="coolwarm"
)

for p in plt.gca().patches:
    height = p.get_height()

    if height > 0:
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width() / 2., height),
            ha='center',
            va='bottom',
            fontsize=9,
            color='black')

plt.title("Fever vs COVID-19", fontsize=14)
plt.xlabel("Fever")
plt.ylabel("Count")
plt.legend(title="COVID-19")
plt.tight_layout()
plt.show()
```



```
[31]: import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))

sns.countplot(
    x='Dry Cough',
    data=df,
    palette="coolwarm"
)

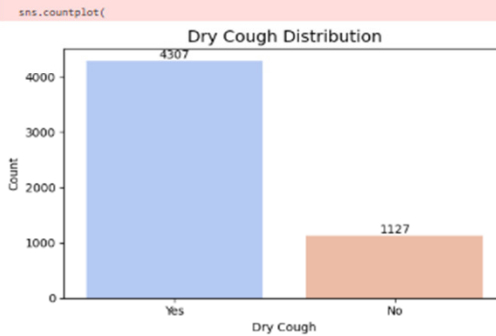
# Add value labels on bars
for p in plt.gca().patches:
    height = p.get_height()

    if height > 0:
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width()/2., height),
            ha='center',
            va='bottom',
            fontsize=10,
            color='black')

plt.title("Dry Cough Distribution", fontsize=14)
plt.xlabel("Dry Cough")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

C:\Users\KrishnaMohan B\AppData\Local\Temp\ipykernel_13264\2676443584.py:6: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.



```
[29]: import matplotlib.pyplot as plt
import seaborn as sns

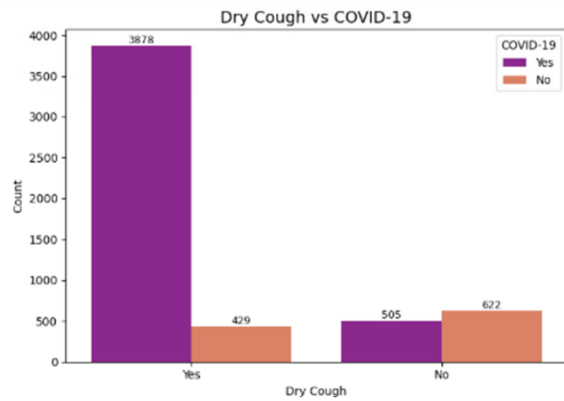
plt.figure(figsize=(7,5))

sns.countplot(
    x='Dry Cough',
    hue='COVID-19',
    data=df,
    palette='plasma'
)

# Add value labels on each bar
for p in plt.gca().patches:
    height = p.get_height()

    if height > 0: # Avoid labeling empty bars
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width()/2., height),
            ha='center',
            va='bottom',
            fontsize=8,
            color='black')

plt.title("Dry Cough vs COVID-19", fontsize=14)
plt.xlabel("Dry Cough")
plt.ylabel("Count")
plt.legend(title="COVID-19")
plt.tight_layout()
plt.show()
```



```
[35]: import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))

sns.countplot(
    x='Sore throat',
    data=df,
    palette='plasma'
)

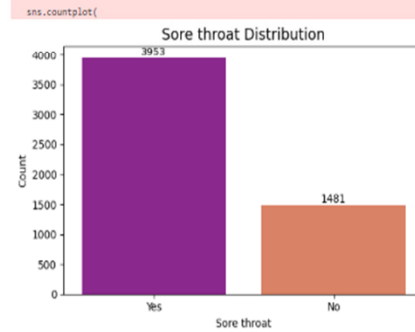
# Add value labels on bars
for p in plt.gca().patches:
    height = p.get_height()

    if height > 0:
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width()/2., height),
            ha='center',
            va='bottom',
            fontsize=10,
            color='black')

plt.title("Sore throat Distribution", fontsize=14)
plt.xlabel("Sore throat")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

C:\Users\KrishnaMohan\B\AppData\Local\Temp\ipykernel_13264\377524078.py:6: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.



```
[37]: import matplotlib.pyplot as plt
import seaborn as sns

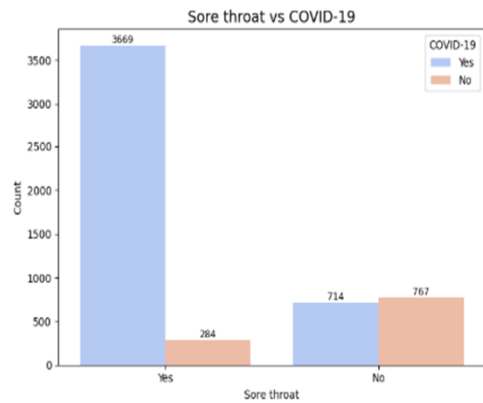
plt.figure(figsize=(7,5))

sns.countplot(
    x='Sore throat',
    hue='COVID-19',
    data=df,
    palette='coolwarm'
)

# Add value labels on each bar
for p in plt.gca().patches:
    height = p.get_height()

    if height > 0: # Avoid labeling empty bars
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width()/2., height),
            ha='center',
            va='bottom',
            fontsize=9,
            color='black')

plt.title("Sore throat vs COVID-19", fontsize=14)
plt.xlabel("Sore throat")
plt.ylabel("Count")
plt.legend(title="COVID-19")
plt.tight_layout()
plt.show()
```



```
[38]: import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))

sns.countplot(
    x='Abroad travel',
    data=df,
    palette='coolwarm'
)

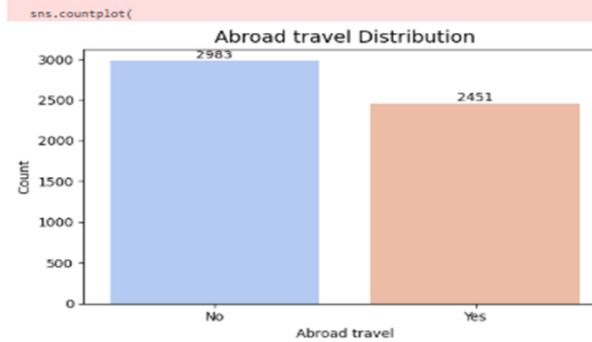
# Add value labels on bars
for p in plt.gca().patches:
    height = p.get_height()

    if height > 0:
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width()/2., height),
            ha='center',
            va='bottom',
            fontsize=10,
            color='black')

plt.title("Abroad travel Distribution", fontsize=14)
plt.xlabel("Abroad travel")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

C:\Users\KrishnaMohan @AppData\Local\Temp\ipykernel_13264\3387310782.py:6: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.



```
[39]: import matplotlib.pyplot as plt
import seaborn as sns
```

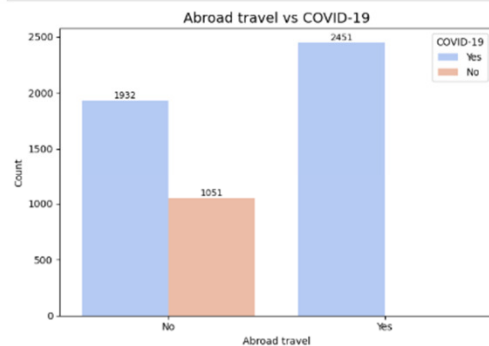
```
plt.figure(figsize=(7,5))

sns.countplot(
    x='Abroad travel',
    hue='COVID-19',
    data=df,
    palette='coolwarm'
)

# Add value labels on each bar
for p in plt.gca().patches:
    height = p.get_height()

    if height > 0: # Avoid labeling empty bars
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width()/2., height),
            ha='center',
            va='bottom',
            fontsize=9,
            color='black')

plt.title("Abroad travel vs COVID-19", fontsize=14)
plt.xlabel("Abroad travel")
plt.ylabel("Count")
plt.legend(title="COVID-19")
plt.tight_layout()
plt.show()
```



```
[40]: import matplotlib.pyplot as plt
import seaborn as sns
```

```
plt.figure(figsize=(6,4))

sns.countplot(
    x='Contact with COVID Patient',
    data=df,
    palette='plasma'
)

# Add value labels on bars
for p in plt.gca().patches:
    height = p.get_height()

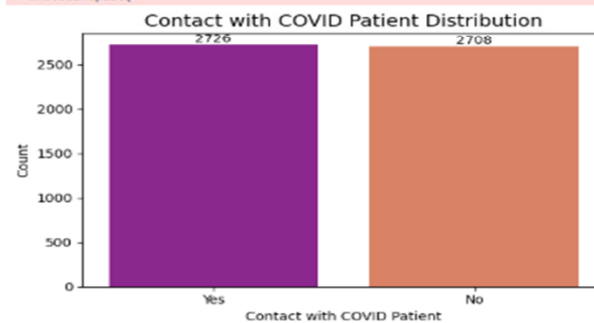
    if height > 0:
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width()/2., height),
            ha='center',
            va='bottom',
            fontsize=10,
            color='black')

plt.title("Contact with COVID Patient Distribution", fontsize=14)
plt.xlabel("Contact with COVID Patient")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

C:\Users\KrishnaMohan 8\AppData\Local\Temp\ipykernel_13264\3024998340.py:6: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.countplot(
```



```
[41]: import matplotlib.pyplot as plt
import seaborn as sns

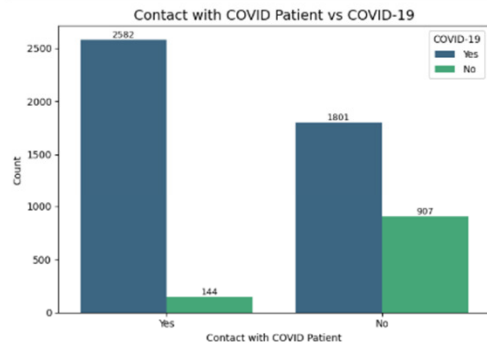
plt.figure(figsize=(7,5))

sns.countplot(
    x='Contact with COVID Patient',
    hue='COVID-19',
    data=df,
    palette="viridis"
)

# Add value Labels on each bar
for p in plt.gca().patches:
    height = p.get_height()

    if height > 0: # Avoid Labeling empty bars
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width()/2., height),
            ha='center',
            va='bottom',
            fontsize=9,
            color='black')

plt.title("Contact with COVID Patient vs COVID-19", fontsize=14)
plt.xlabel("Contact with COVID Patient")
plt.ylabel("Count")
plt.legend(title="COVID-19")
plt.tight_layout()
plt.show()
```



```
[43]: import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))

sns.countplot(
    x='Attended Large Gathering',
    data=df,
    palette="coolwarm"
)

# Add value Labels on bars
for p in plt.gca().patches:
    height = p.get_height()

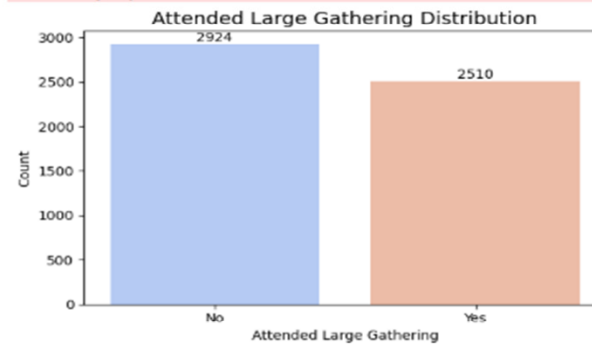
    if height > 0:
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width()/2., height),
            ha='center',
            va='bottom',
            fontsize=10,
            color='black')

plt.title("Attended Large Gathering Distribution", fontsize=14)
plt.xlabel("Attended Large Gathering")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

C:\Users\KrishnaMohan B\AppData\Local\Temp\ipykernel_13264\2084580446.py:6: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.countplot(
```



```
[44]: import matplotlib.pyplot as plt
import seaborn as sns

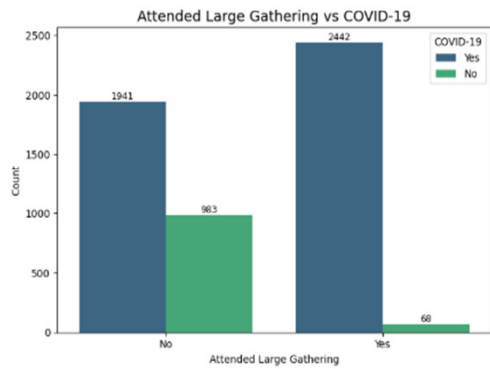
plt.figure(figsize=(7,5))

sns.countplot(
    x='Attended Large Gathering',
    hue='COVID-19',
    data=df,
    palette='viridis'
)

# Add value labels on each bar
for p in plt.gca().patches:
    height = p.get_height()

    if height > 0: # Avoid labeling empty bars
        plt.annotate(f'{int(height)}',
            (p.get_x() + p.get_width()/2., height),
            ha='center',
            va='bottom',
            fontsize=9,
            color='black')

plt.title('Attended Large Gathering vs COVID-19', fontsize=14)
plt.xlabel('Attended Large Gathering')
plt.ylabel('Count')
plt.legend(title='COVID-19')
plt.tight_layout()
plt.show()
```

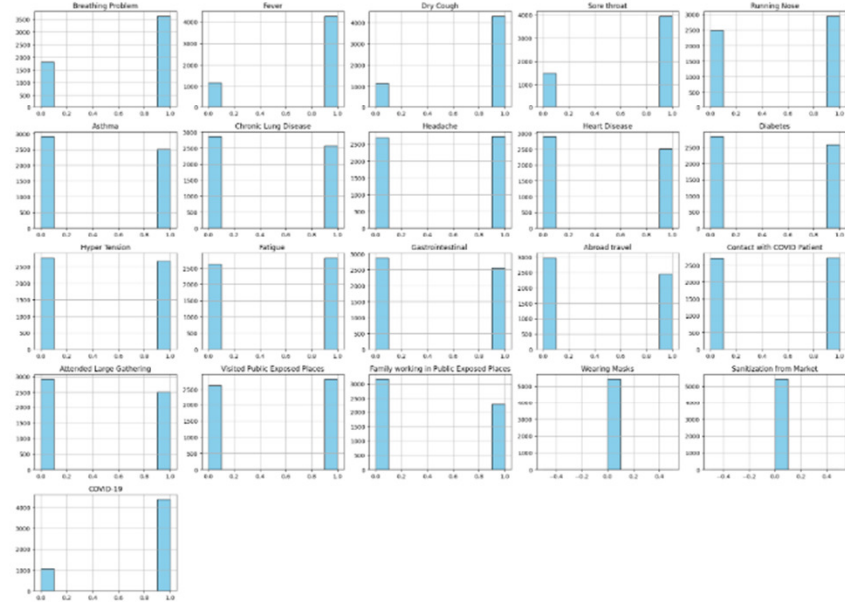


```
[52]: import pandas as pd
import matplotlib.pyplot as plt
plt.figure(figsize=(20,15))

df.select_dtypes(include=['int64','float64']).hist(
    figsize=(20,15),
    color='skyblue',
    edgecolor='black'
)

plt.tight_layout()
plt.show()
```

<Figure size 2000x1500 with 0 Axes>



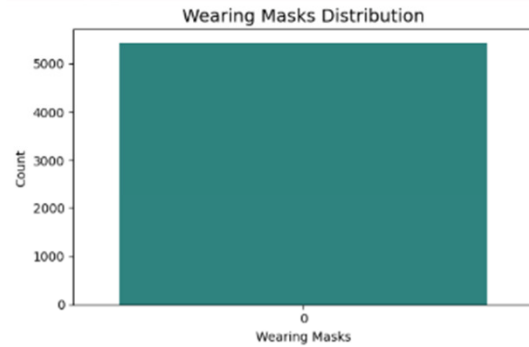
```
[56]: import matplotlib.pyplot as plt
import seaborn as sns
print(df['Wearing Masks'].value_counts())
plt.figure(figsize=(6,4))
sns.countplot(
    x='Wearing Masks',
    data=df,
    palette='viridis'
)
plt.title("Wearing Masks Distribution", fontsize=14)
plt.xlabel("Wearing Masks")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

```
Wearing Masks
0    5434
Name: count, dtype: int64
```

C:\Users\KrishnaMohan @\AppData\Local\Temp\ipykernel_13264\3896807302.py:5: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.countplot(
```



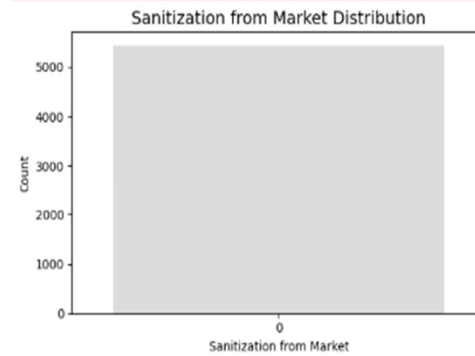
```
[58]: import matplotlib.pyplot as plt
import seaborn as sns
print(df['Sanitization from Market'].value_counts())
plt.figure(figsize=(6,4))
sns.countplot(
    x='Sanitization from Market',
    data=df,
    palette='coolwarm'
)
plt.title("Sanitization from Market Distribution", fontsize=14)
plt.xlabel("Sanitization from Market")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

```
Sanitization from Market
0    5434
Name: count, dtype: int64
```

C:\Users\KrishnaMohan @\AppData\Local\Temp\ipykernel_13264\3515519473.py:5: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.countplot(
```

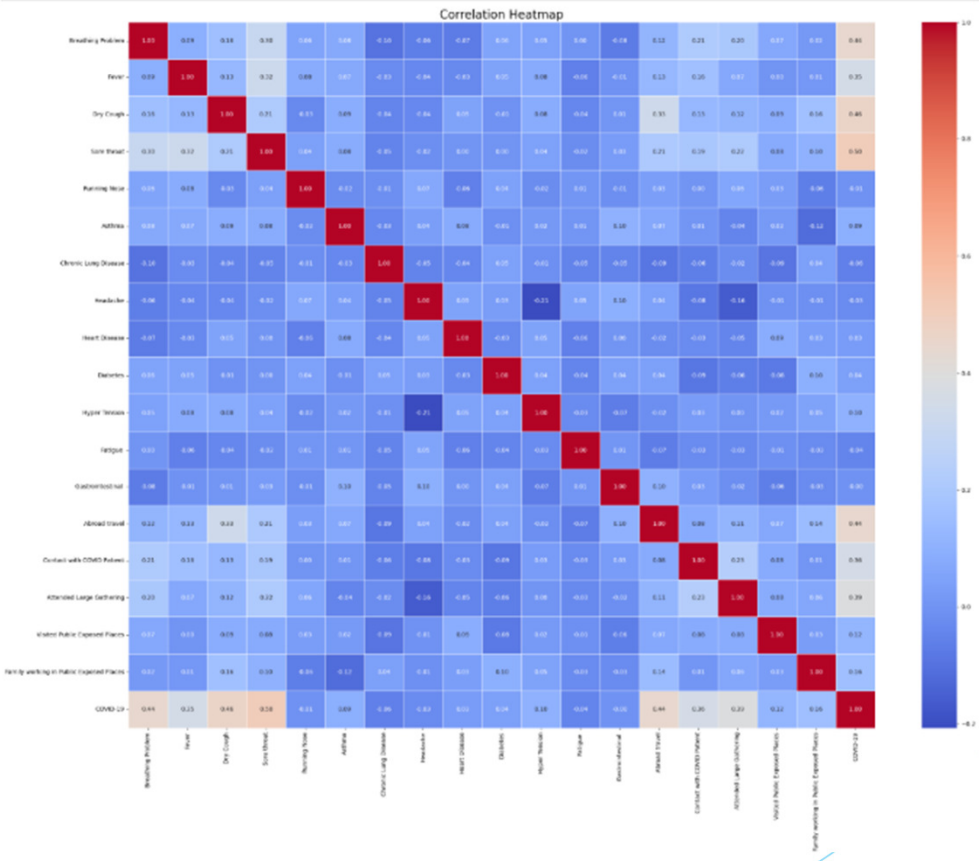


```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(25,20))

sns.heatmap(
    df_corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5,
    linecolor='white'
)

plt.title("Correlation Heatmap", fontsize=20)
plt.tight_layout()
plt.show()
```



Conclusion

The COVID-19 prediction system developed using machine learning leverages a set of clinical symptoms, pre-existing health conditions, lifestyle factors, and exposure risks—including breathing problems, fever, dry cough, sore throat, running nose, asthma, chronic lung disease, headache, heart disease, diabetes, hypertension, fatigue, gastrointestinal issues, travel history, contact with COVID patients, attendance at large gatherings, visits to public-exposed places, family exposure, mask usage, and market sanitization habits—to accurately predict the likelihood of infection. By applying machine learning algorithms to this dataset, the system is able to identify patterns and correlations between symptoms, health conditions, and exposure factors, allowing for timely risk assessment. This approach aids in early detection and preventive measures, helping individuals and healthcare authorities take informed actions to reduce the spread of COVID-19. Overall, the study demonstrates that machine learning models can serve as effective predictive tools in public health, complementing traditional testing methods and supporting data-driven decision-making during pandemic situations.

Future Scope

The COVID-19 prediction system, which uses symptoms (breathing problems, fever, dry cough, sore throat, running nose, fatigue, headache, gastrointestinal issues), pre-existing conditions (asthma, chronic lung disease, heart disease, diabetes, hypertension), and exposure factors (abroad travel, contact with COVID patients, attendance at large gatherings, visits to public-exposed places, family exposure, mask usage, and market sanitization), has significant potential for future enhancement. Future improvements could include integrating real-time health monitoring through wearable devices or mobile apps, enabling early alerts for high-risk individuals. Models could be extended to predict disease severity, hospitalization needs, and recovery outcomes, helping healthcare systems allocate resources efficiently. Additionally, incorporating larger, more diverse datasets and analyzing patterns related to emerging variants can enhance predictive accuracy. Overall, this approach can support data-driven public health strategies, early intervention, and pandemic preparedness for future outbreaks.

References

- ▶ <https://www.python.org/>
- ▶ <https://scikit-learn.org/stable/>
- ▶ <https://flask.palletsprojects.com/en/stable/>

Thank You

